

AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems

AIKernel Formal Foundations

Contract-Based Semantic Execution for Governed AI Systems

著者: Takuya Sogawa

ORCID: 0009-0009-7499-2595

Version: v0.1.0

Publication date: 2026-05-31

DOI: 10.5281/zenodo.20460322

正本: English

副本: Japanese

Abstract / 概要

AIKernel は、確率的な AI 推論を、契約ベースの意味的実行へ再構成するための形式的アーキテクチャである。本稿は、LLM の出力を最終的な権威ある回答として扱うのではなく、決定論的な契約の下で受理、観測、統治、変換、記録されるべき状態遷移候補として扱う。

本稿は、統治された AI システムのための Semantic Context Operating System として、AIKernel の基礎理論を定義する。契約ベース実行、意味的軌跡、決定論的ガバナンス、Interface-Led Architecture、Fail-Closed な Result 伝播を中核原則として整理し、知識同一性、セマンティックストレージ、推論前受理統治、軌跡統治、非同期実行トランザクションからなる Phase-1 論文群を一貫した基盤として位置づける。

本稿の貢献は、AIKernel 論文シリーズ、AIKernel.NET 実装方針、および Interface-Led Architecture の方法論を接続する最上位の概念地図と形式的語彙を提示する点にある。本稿は AIKernel が文字通りの OS カーネルであると主張するものではない。むしろ、AI 推論を構造化され、監査可能で、再現可能な計算として扱うために必要な実行意味論、契約境界、統治責務を定義する。

1. Introduction / 序論

AIKernel は、確率的 AI 推論を決定論的なソフトウェアシステムへ統合するためのアーキテクチャおよび理論フレームワークである。その中心的な主張は、AI の振る舞いを、ユーザーとモデルの間の制約のない対話として扱うべきではない、という点にある。AI 推論は、セマンティックランタイム内で発生する、契約によって統治された状態遷移として扱われるべきである。

従来の AI アプリケーションでは、モデルがプロンプトを受け取り、テキストを生成し、周囲のアプリケーションがその結果を解釈する。この設計では、入力 of 受理可能性、能力境界、ポリシー強制、意味的漂流制御、構造化エラー伝播、監査可能なリプレイといった重要な責務が暗黙のまま残る。

本稿は、AIKernel の最上位の形式的基礎を定義し、以下の 6 つの転換を明示する。

1. AI 推論を状態遷移系として扱う。
2. 推論をアドホックなプロンプト交換ではなく契約として表現する。
3. 確率的な出力候補は決定論的ガバナンス境界を通じてのみ受理される。

4. Interface-Led Architecture (ILA) を AI ランタイム設計へ適用する。
5. Semantic Trajectory を観測と制御の第一級対象とする。
6. AIKernel.NET をこれらの抽象の参照ランタイムとして位置づける。

その結果、AI システムは不透明な確率的サービスではなく、統治された意味的実行環境として扱えるようになる。

2. Core Principles of AIKernel / AIKernel の基礎原則

2.1 Contract-Based Execution / 契約ベース実行

すべての AI 推論プロセスは、単なる API 呼び出しではなく契約として表現される。契約は、推論トランザクションが開始、継続、出力、失敗する条件を定義する。

最低限、実行契約は以下を含む。

- **Preconditions:** 実行前に満たすべき入力構造、権限、能力、文脈要件。
- **Admissible states:** ガバナンス層が受理する文脈状態の範囲。
- **Invariants:** 実行中および実行後に破壊してはならない制約。
- **Postconditions:** 検証済み出力に対する構造的・意味的保証。
- **Failure semantics:** 契約違反、拒否、異常、失敗に対する決定論的処理。

この考え方は Design by Contract の伝統を継承しつつ、生成された出力そのものを、検証前には契約保存的な結果として信頼できない AI システムへ拡張するものである。

2.2 Semantic Trajectory / 意味的軌跡

AIKernel は推論を単一のテキスト応答として扱わない。推論は、意味的状态の列として扱われる。

$$\text{Trajectory} = (State_0 \rightarrow State_1 \rightarrow \dots \rightarrow State_n)$$

意味的軌跡は、漂流し、収束し、分岐し、崩壊し、ガバナンス境界を破ることがある。このため、AI の振る舞いを言語表現だけではなく、幾何学的・手続き的に評価できる。

Trajectory Governance の目的は、モデルの隠れた思考を読むことではない。外部から表現可能な意味状態、候補行動、リスク信号、契約遷移を観測することである。

2.3 Deterministic Governance / 決定論的ガバナンス

確率的モデルは提案する。決定論的カーネルは統治する。

AIKernel はモデル生成とガバナンス判断を分離する。モデル出力は遷移候補として扱われ、ランタイムがポリシー、文脈、能力、リスク制約の下で受理可能かどうかを判断する。

ガバナンスモデルには以下が含まれる。

- **Pre-Inference Admissibility:** モデル実行前の評価。
- **Trajectory Governance:** 実行中の遷移列の評価。
- **Policy Enforcement:** セキュリティおよび組織制約の決定論的適用。
- **Safety Constraints:** 安全でない、または検証不能な遷移の Fail-Closed な隔離。

2.4 Interface-Led Architecture / ILA

AIKernel は Interface-Led Architecture の応用である。構成要素は、実装詳細ではなく、インターフェース、契約、能力、失敗意味論から先に定義される。

実装は、確率的、外部的、非決定的、リモート、あるいは Provider 固有であり得る。一方で、インターフェースは決定論的で、監査可能で、契約保存的でなければならない。

AI システムにおいて、モデル Provider がガバナンス権限を持つてはならない。ガバナンスは確率的生成器の外側に存在する必要がある。

2.5 Fail-Closed Execution / Fail-Closed 実行

Fail-Closed 実行とは、不確実、無効、拒否済み、不正形式、または検証不能な結果を、有効な結果としてパイプラインへ流さないことである。予測可能なドメイン失敗は、制御不能な例外ではなく、`Result<T>` のような構造化された値として伝播される。

予期しないインフラ障害は例外的であり得るが、ガバナンス境界を黙って通過してはならない。Fail-Closed な AI ランタイムは、契約保存的な結果を返すか、安全な失敗状態で停止する。

3. Semantic State Model / 意味的状态モデル

AIKernel は意味的実行を、Structural Layer、Governance Layer、Execution Layer の三層でモデル化する。

3.1 Structural Layer / 構造層

構造層は、文脈を組み立てるための安定した基盤を定義する。

- **ROM Snapshot:** 正準化された不変知識単位。
- **VFS Semantic Storage:** セマンティックストレージに対する決定論的アクセス境界。
- **Context Graph:** 知識、状態、ポリシー、ランタイム入力の検証済み依存関係。

この層は「どの状態を、どの同一性の下で読むのか」に答える。

3.2 Governance Layer / 統治層

統治層は、遷移が進行してよいかどうかを判断する。

- **Admissibility:** リクエスト、制約、能力の推論前受理。
- **Policy and Risk:** 組織制約と実行リスクの評価。
- **Safety:** 意図、文脈、権限が危険または曖昧な場合の拒否または明確化要求。

この層は「この遷移は許容されるか」に答える。

3.3 Execution Layer / 実行層

実行層は、統治された計算と状態変換を実行する。

- **Provider:** モデル出力、埋め込み、ストレージ値、時刻、ツール結果、その他の能力を供給する。
- **Observer:** 状態、メトリクス、リスク、漂流、トークン使用量、監査信号を記録する。
- **Operator:** 検証済み状態を次の契約保存的な状態へ変換する。
- **Result Pipeline:** `Task<Result<T>>` または同等の Fail-Closed 値によって非同期実行を合成する。

この層は「受理された遷移をどのように安全に実行するか」に答える。

4. AIKernel Execution Model / 実行モデル

4.1 Provider / Observer / Operator

AIKernel は Provider-Observer-Operator 抽象化規律をランタイム設計に適用する。

- **Provider:** 値または能力を供給する。モデル Provider はテキストを生成し、Embedding Provider はベクトルを生成し、VFS Provider は保存データを供給する。Provider はガバナンス判断を行わない。

- **Observer:** 証拠を測定、記録、報告する。Observer は状態を直接変更しない。
- **Operator:** 状態を変換し、受理済み操作を実行し、次状態を構築する。Operator はシステム状態を変更し得るため、統治されなければならない。

上位ランタイム構造は、これらの基本役割から構成される Unit として表現される。Unit は Pipeline によって連結され、Core Contract の下でのみ動作する。

4.2 Execution Contract / 実行契約

すべての実行フェーズは、以下の特性を維持しなければならない。

1. **Deterministic:** 同一の受理済み入力とリプレイデータに対して、契約レベルでは同じ経路を辿る。
2. **Fail-Closed:** 不確実または無効な結果を有効な出力として受理しない。
3. **Contract-Preserving:** 事前条件、不変条件、事後条件を維持する。
4. **Observable:** 関連するすべての遷移が証拠を生成する。
5. **Replayable:** ReplayLog または同等の監査記録によって実行を再構成できる。

4.3 Result Monad / Result モナド

AIKernel は予測可能な失敗を値としてルーティングする。C# では次のように表現できる。

```
Task<Result<T>>
```

Result Pipeline により、拒否、パース失敗、検証失敗、ポリシー失敗は構造化された値としてパイプライン上を伝播する。これにより、予測可能な AI failure が制御不能な例外へ変化することを防ぎ、決定論的な実行フローを支える。

5. Governance Model: Phase-1 Overview / Phase-1 概要

AIKernel Phase-1 は、最小のガバナンスおよび実行基盤を定義する 5 本の中核論文で構成される。

1. **Paper 01 (ROM Format & Knowledge Snapshot Model)** - 不変知識の同一性と正準スナップショット。
2. **Paper 02 (VFS Architecture & Semantic Storage Model)** - セマンティックストレージ境界と Capability-based access。
3. **Paper 03 (Pre-Inference Admissibility Governance)** - 権限、制約、計算限界の推論前審査。
4. **Paper 04 (Trajectory Governance Model)** - 意味的軌跡の実行時監視と制御。
5. **Paper 05 (Async Result Pipeline & Execution Transaction Model)** - Task<Result<T>> による決定論的パイプラインと、SGP モデルを含む推論トランザクション。

これらの論文は、AI 推論を統治されたプロセスとして展開するための基本条件を定義する。

6. Semantic Trajectory and State Transitions / 意味的軌跡と状態遷移

Trajectory Governance は、AI 推論を意味的状态空間における運動として一般化する。中心概念は以下である。

- **Semantic Ellipsoid:** 許容される意味領域の運用上の近似。
- **Drift:** RootGoal または受理済み意味意図からの逸脱。
- **Convergence:** 安定した契約保存的な目標状態へ向かう運動。
- **Risk Surface:** ツール、権限、行動、外部効果によって生じるリスク構造。

これらの概念は、人間の意味を完全に計量するものではない。AI 遷移を統治するための、運用上の観測プリミティブである。

7. Formalization Sketch / 形式化の方向性

AIKernel は、統治された状態遷移系として形式化できる。

遷移は次のように書ける。

$$S_{t+1} = \delta(S_t, A_t)$$

ここで、 S_t は現在の意味状態であり、 A_t は提案された行動または遷移候補である。ガバナンスは、次の受理判定を導入する。

$$G(S_t, A_t, C_t, P_t) \in \{\text{Permit, Deny, Clarify, Abort}\}$$

ここで、 C_t は文脈、 P_t はポリシーを表す。

今後の形式化は以下を含む。

1. **State transition systems:** 確率的候補生成を包む決定論的ガバナンス。
2. **Contract algebra:** 事前条件、不変条件、事後条件の合成。
3. **Trajectory measures:** 操作的距離、drift metrics、不確実性近似。
4. **Formal safety logic:** Fail-Closed 性質のための時相論理または様相論理。

8. Relation to Phase-2: Semantic Compilation / Phase-2 への接続

本稿は、意味的遷移を安全に統治し実行するための基盤を定義する。Phase-2 である Semantic Compilation Architecture は、ガバナンス境界から Semantic Compilation へ焦点を移す。

Phase-2 では、人間の意図と構造化文脈が、統治された意味的中間表現と回路候補へ変換される。中心的な問いは「この遷移は受理可能か」から、「受理された意味構造を、いかに検証可能で統治された実行トポロジーへコンパイルできるか」へ移る。

したがって、Phase-0 と Phase-1 は基盤を提供し、Phase-2 はその上に構築される意味論的ランタイム理論を定義する。

9. Conclusion / 結論

AIKernel Formal Foundations は、AI 推論をブラックボックスな prompt-response pattern から、状態、軌跡、統治、実行にまたがる契約ベースの計算へ再構成する。

Interface-Led Architecture、Provider-Observer-Operator の役割分離、Fail-Closed Result Pipeline により、確率的モデル出力は最終権威ではなく、遷移候補として扱われる。

AIKernel は、契約・状態・軌跡・統治を中核とする Semantic Context Operating System の基礎理論として機能する。

Appendix A: AIKernel Paper Series Overview / 論文シリーズ概要

- **Phase-0:** Foundational Theory (本稿)
- **Phase-1:** Semantic Context OS Governance & Execution (Paper 01-05)
- **Phase-2:** Semantic Compilation Architecture
- **Extensions:** TensorKernel and domain-specific governance layers

Appendix B: Execution Contract Examples / 実行契約の例

AIKernel の実行契約は、以下の抽象フローとして表現できる。

1. **Provider Routing:** model、storage、tool provider への capability-based delegation。
2. **Governance Flow:** 決定論的ガバナンスポリシーによる推論前および軌跡評価。
3. **Result Pipeline:** Task<Result<T>> による Structure -> Generation -> Polish (SGP) のモナド的合成。

Appendix C: Relation to ILA / POO / PSM

本基礎理論は、以下の三つの補完的アーキテクチャ思想を接続する。

- **ILA (Interface-Led Architecture):** 契約優先のソフトウェア方法論。
- **POO (Provider-Observer-Operator):** ILA を適用するための役割ベース抽象化規律。
- **PSM (Prompt-State Machine):** LLM 推論を構造化するためのプロンプト空間上の状態遷移モデル。

References / 参考文献

1. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model."Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
2. Sogawa, Takuya. "Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture."Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
3. Sogawa, Takuya. "Prompt-State Machines: Applying Interface-Led Architecture to Structure LLM Reasoning."Zenodo, 2026. DOI: 10.5281/zenodo.20323512.
4. Sogawa, Takuya. "AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model."Zenodo, 2026. DOI: 10.5281/zenodo.20306539.
5. Sogawa, Takuya. "AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model."Zenodo, 2026. DOI: 10.5281/zenodo.20307891.
6. Sogawa, Takuya. "AIKernel Phase-1 Paper 03: Pre-Inference Admissibility Governance."Zenodo, 2026. DOI: 10.5281/zenodo.20308639.
7. Sogawa, Takuya. "AIKernel Phase-1 Paper 04: Trajectory Governance Model."Zenodo, 2026. DOI: 10.5281/zenodo.20309510.
8. Sogawa, Takuya. "Asynchronous Result Pipelines in C#: Task<Result> and LINQ-Based Monadic Composition for AI Applications."Zenodo, 2026. DOI: 10.5281/zenodo.20458492.
9. Sogawa, Takuya. "AIKernel Phase-2 Theory: Semantic Compilation Architecture."Zenodo, 2026. DOI: 10.5281/zenodo.20312092.
10. Hoare, C. A. R. "An Axiomatic Basis for Computer Programming."Communications of the ACM, vol. 12, no. 10, 1969, pp. 576-580. DOI: 10.1145/363235.363259.
11. Meyer, Bertrand. Object-Oriented Software Construction. 2nd ed., Prentice Hall, 1997.
12. Lamport, Leslie. "The Temporal Logic of Actions."ACM Transactions on Programming Languages and Systems, vol. 16, no. 3, 1994, pp. 872-923. DOI: 10.1145/177492.177726.